RATER Competition

Problem description

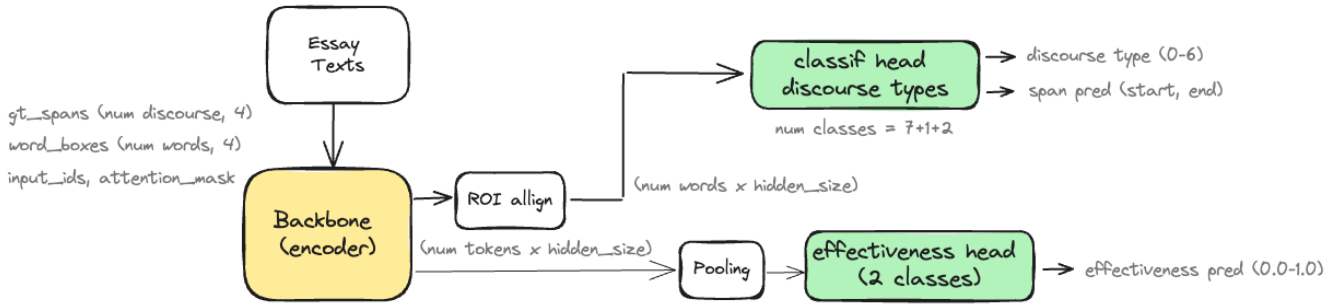
The aim of this challenge is to create more efficient versions of the algorithms developed in the original Feedback Prize competition series. The task will be to develop an efficient "super-algorithm" that can:

- Segment an essay into individual argumentative elements and classify each element according to its argumentative type (similar to the "Feedback Prize - Evaluate Student Writing" Kaggle competition task), and
- Evaluate the effectiveness of each discourse element (similar to the "Feedback Prize - Predicting Effective Arguments" Kaggle competition task)

Submissions are scored using a custom evaluation metric [4]. You can read more about the evaluation metric here.

Solution Summary [TLDR]

My solution consists of a multi-task model (with a single backbone) that handles both span prediction, discourse elements classification and discourse effectiveness predictions. The problem is treated as an object detection task, as first introduced in FP1 competition by tascj [1]. No pre-processing or extensive post-processing is applied.



Backbones used:

- deberta-base (for early experiments)
- deberta-large
- deberta-xlarge

Final solution:

- 1 x seed of deberta-large trained with all data (SWA checkpoint as a result of averaging weights from last 2 epochs) plus
- 1 x seed of deberta-large (single fold)
- 1 x seed of deberta-xlarge trained with all data

Inference time: approx. 15 mins (per checkpoint)

LB: 0.6029 (single model, deberta-large)
CV (without overlap removal):

```
comp_score = 0.6867
comp_score = 0.7180 (single weighted)
FP1_score  = 0.7588 (old competition metric)
```

CV (after overlap removal):

```
comp_score = 0.6329
comp_score = 0.6779 (single weighted)
FP1_score  = 0.7170 (old competition metric)
```

Ensemble LB: 0.6210 (3 checkpoint with WBF)

Modelling

As described above the model is based on [1] with several modifications outlined below.
Initial weights by [5], freeze half layers.

Classes

- 1 for objectness
- 2 for regression (span start, end)
- 7 classes for discourse type classification
- 2 classes for discourse effectiveness

Target constructed as in [1] with required modifications for effectiveness predictions (e.g. add extra positive, see the model subclass `build_target()` for better understanding)

Loss: `w1 * (obj_loss + reg_loss + classif_loss) + w2 * (effectiveness_loss)`

where,

`obj_loss:` BCE loss for objectness
`reg_loss:` iou log loss for span prediction
`classif_loss:` BCE loss for discourse elements classification
`effectiveness_loss:` BCE loss for discourse effectiveness

Loss weights w_1, w_2 were varied in different experiments, typical values were, w_1=1.0, w_2= 0.5-2.0
Post-process: only NMS inside the prediction loop

Training

- 5fold cross-validation stratified by discourse effectiveness
- Freeze half layers from backbone
- Batch size = 1
- Gradient accumulation steps = 1 – 4
- Learning rate: 4e-5
- Warm-up: 0.05
- Averaging weights during last few epochs
- modified WBF method (to account for effectiveness predictions)

Inference-Submission

Instructions to run the inference code are given in the README.md file.

Other experiments

- NER approach with 15 classes and the usual BIO-tagging, as described in FP1 1st and 2nd solutions [2, 3] and add an extra head with 3 classes (Non-Effective, Effective, None) for effectiveness predictions. Best LB scores with this approach were 0.58x with a deberta-large with bi-LSTM head.
- Object detection approach with 14 (+1+2) classes, as described in FP1 6th solution [1]. Best LB scores with this approach were 0.58x with a deberta-large backbone.

- Stacking for effectiveness predictions could improve approx. 0.005
- Ensembling with WBF method (modified to account for effectiveness predictions) could improve scores but I'd expected a better performance.
- Unfortunately, I didn't have the time for knowledge distillation experiments, or to explore post-processing ideas.

Things didn't work

- I didn't have the time to improve the scores with deberta-v3 variants. In my early expriments were approx. 0.1 bellow. Similar with Longofrmer, although results were slightly better.
- Use embeddings from LLM (e.g. Mistral/Mixtral)

Acknowledgements

I'd like to thank the Learning Agency team for the great support throughout the whole duration of the competition and all kagglers for sharing their work in previous Feedback competitions.

References

[1] 6th place solution. A YOLO-like text span detector: https://www.kaggle.com/competitions/feedback-prize-2021/discussion/313424

[2] Second Place Solution: https://www.kaggle.com/competitions/feedback-prize-2021/discussion/313389

[3] 1st Solution : https://www.kaggle.com/competitions/feedback-prize-2021/discussion/313177

[4] Shovel-Ready Competition Unified Metric, Ralph Abboud

[5] https://www.kaggle.com/datasets/tascj0/feedback-checkpoints/data